FilsPay

Submission document

Audit, fixes, deployment and capacity · 30 October 2026

Team

TeamName

Member	What they owned
Member 1	Owner: ...
Member 2	Owner: ...
Member 3	Owner: ...

Live URLs

Checkout	https://fp-pay-tzakir.dilmune.app/pay/<token>
Merchant dashboard	https://fp-pay-tzakir.dilmune.app/dashboard/login
Admin console	https://fp-admin-tzakir.dilmune.app/login
Public API	https://fp-api-tzakir.dilmune.app (health: /healthz, /readyz; API: /v1)

Repository

Public fork	https://github.com/thrzkr/fils-pay
Last commit	`64a665eace45f71dfb52a7c003f751c2a1f2ff1f`
Branch	`main` (default branch of the fork)

Demo merchant login (shared, simulated money): merchant ID `94850160-a3f5-4377-a260-af4fe28e7743`, `demo@filspay.demo` / `FilsPayDemo!2026`. The login page shows the one-time code (see finding F-25).

Contents

1. Summary 3

2. Architecture and deployment 3

2.1 Redeploy in one command . . . 4

2.2 Where secrets come from . . . 4

2.3 Health checks and restart policy . . . 5

2.4 How a redeploy avoids breaking a settlement run . . . 5

3. Capacity 5

3.1 Our load test . . . 5

3.2 Where the bottleneck is . . . 6

3.3 Turning away excess load and recovering . . . 6

4. Findings report 6

4.1 Findings . . . 6

4.2 Root causes . . . 12

4.3 Decisions and open risks . . . 13

5. UI and UX changes 13

5.1 Refund control on the transaction page . . . 13

5.2 Transaction page no longer crashes after a refund . . . 14

5.3 Site root and unknown paths . . . 14

5.4 Refund and settlement views after the demo . . . 14

6. AI usage 15

6.1 Setup . . . 15

6.2 Excerpt 1: an inherited test that asserted the bug . . . 15

6.3 Excerpt 2: the AI was wrong, and we caught it . . . 16

6.4 Excerpt 3: the AI's own report was wrong; a browser run showed it . . . 16

Appendix 16

A. Test output (fresh Postgres 16, 30 Oct 2026) . . . 16

B. Refund demo evidence (local, 2026-10-30) . . . 16

C. Migrations added . . . 17

D. Reproduce . . . 17

1. Summary

What we found. FilsPay worked on the happy path and failed wherever money met concurrency, tenancy or rounding. We logged **26 code findings** and **24 deployment/operations bugs**. The worst were: the login code (OTP) could be skipped entirely; one merchant's password could open another merchant's account; refunds returned 501 and, once wired, could pay out twice; finance CSV adjustments were posted 10x too small; webhooks were signed over different bytes than were sent; the settlement worker duplicated payout notifications; and the hosted API site served the git checkout, including an admin password, as static files.

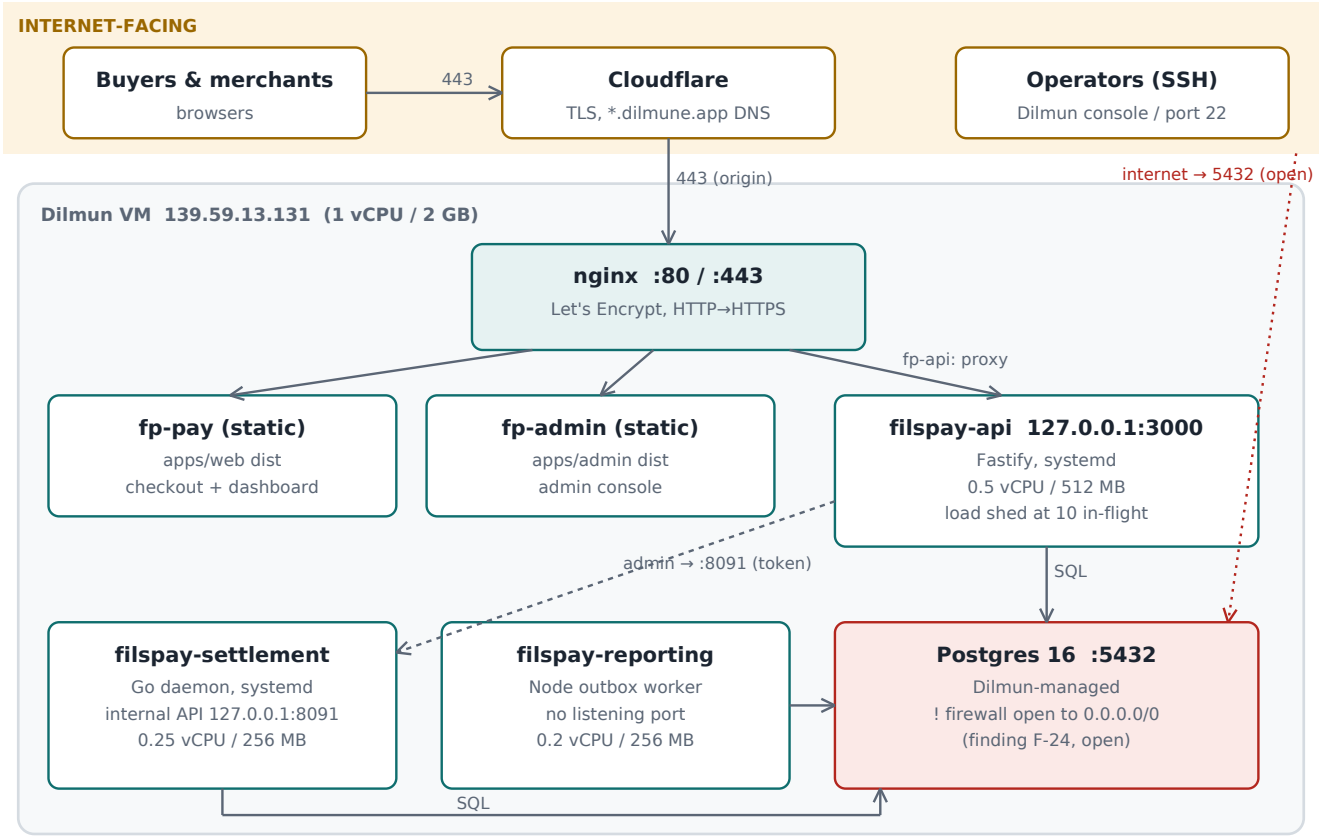
What we fixed. 24 of the 26 code findings are fixed. Each has a test that fails on the old code, or a query or live request as proof. We added refunds end to end (API, row-locked reservation, fee reversal, webhook, dashboard drawer), made payment links safe against double charges, moved all money paths to integer fils, and deployed the stack to Dilmun Cloud behind HTTPS with systemd restarts, load shedding and a one-command redeploy. Fresh run on a clean Postgres: **TypeScript 453/453, Go all packages pass**.

What we left alone. We did not change settlement to net refunds out of payouts (finding F-01). Finance owns that decision, and a change to payout logic on the deadline day is riskier than the bug. We also did not rewrite git history to purge committed secrets, did not run backfills on production data, and did not add an email/SMS sender for OTP codes. All of these are in section 4.3 with the safeguard we propose.

Main root cause. The team treated generated code, code comments and tests as the source of truth instead of the ledger. Tests were written to assert bugs, and "Claude says" comments closed questions without a check. So the same four patterns repeated across the codebase: **check-then-act without a lock**, **two sources of truth for one number**, **money not typed as integer fils end to end**, and **queries not scoped to the caller's merchant**.

2. Architecture and deployment

One Dilmun Cloud VM (1 vCPU, 2 GB RAM, 50 GB) behind Cloudflare. nginx and Postgres 16 are managed by Dilmun. The three FilsPay processes run as systemd services. Only nginx (80/443) is meant to face the internet.



Ports: **443**/**80** (nginx, public through Cloudflare); **22** (SSH, operators); **3000** API and **8091** settlement internal API bind to 127.0.0.1 only (verified closed from outside); **5432** Postgres is still reachable from the internet (finding F-24, open). Services talk through the shared database and the transactional outbox.

2.1 Redeploy in one command

After Dilmun pulls the new commit into the fp-api site, one idempotent command rebuilds and restarts everything:

```
bash ops/dilmune/setup.sh
```

- Fails fast if any required variable is missing from `.env`.
- Applies database migrations (`db/migrate.ts`, already-applied files are skipped).
- Builds the Go settlement binary (installs Go 1.22 if absent).
- Writes and (re)starts three systemd units with CPU and memory caps, takes the API over from the old pm2 process.
- Exits non-zero unless all three services are `active` and `/readyz` answers within 30 s.

Frontends are static builds on their own Dilmun sites (build commands in `docs/DEPLOY.md`; the API URL is baked in at build time and checked with a bundle grep after deploy). Local equivalent: `docker compose -f ops/docker-compose.yml up -d --build`.

2.2 Where secrets come from

- Secrets live only in the fp-api site's **Environment variables** in Dilmun. Dilmun writes them to `.env` in the site root on deploy. `setup.sh` sets it to `chmod 600` and systemd loads it with `EnvironmentFile=`.
- `.env*` is gitignored. Nothing secret is in `setup.sh`, compose or the repo. The `.claude/settings.json` deny rules stop the assistant from reading them.
- Production refuses to boot without `WEBHOOK_SECRET_ENCRYPTION_KEY` (32 bytes), `ACQUIRER_CALLBACK_SIGNING_SECRET` and `SETTLEMENT_INTERNAL_TOKEN` (`config.ts`, tested).
- **Exposure:** an admin password, `merchant-details.json` and the sandbox acquirer secret in `ops/.env.docker` were committed earlier. They are untracked now but still in history; they must be rotated (section 4.3).

2.3 Health checks and restart policy

Check / policy	Detail
`GET /healthz`	Process is up (never shed by the load-shedding plugin).
`GET /readyz`	Database reachable. Used by `setup.sh` and by our post-deploy checks.
`GET /`	Service JSON with links (was a 404).
systemd	`Restart=always`, `RestartSec=2`, `StartLimitIntervalSec=0` (never gives up), `MemoryMax` with `MemorySwapMax=0` so an OOM kill restarts cleanly; `KillMode=control-group` also stops tsx's child process.
Boot	All three units are `enabled` and start after `postgresql.service`.

2.4 How a redeploy avoids breaking a settlement run

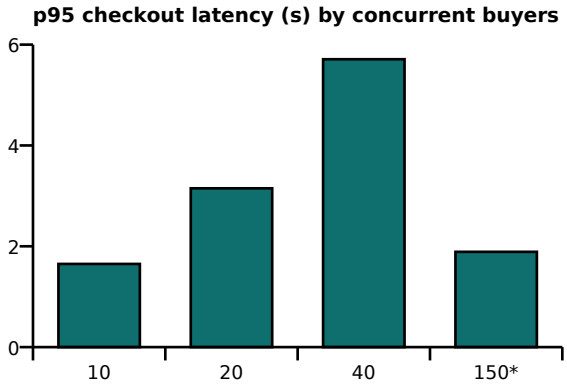
- A restart sends SIGTERM. The daemon wires it to a cancelled context (`signal.NotifyContext`), stops taking new work and lets the in-flight Postgres transaction commit or roll back. `TimeoutStopSec=30` gives it time before a hard kill.
- Each batch step (`CloseBatch`, `PayoutBatch`) is **one Postgres transaction** under a `FOR UPDATE` lock on the batch row, so a kill can never leave a half-closed batch.
- On start, the daemon immediately re-runs the latest completed Bahrain business day. Closed or paid batches are no-ops (unique `(merchant_id, batch_date)`), so an interrupted run finishes on restart and a completed one is not repeated.
- Payout notifications are guarded by a unique index (migration 0076), so a restart that overlaps a CLI run cannot double-notify.
- **Operational rule:** do not redeploy between 00:00 and 01:00 Bahrain time, when the daily close runs.

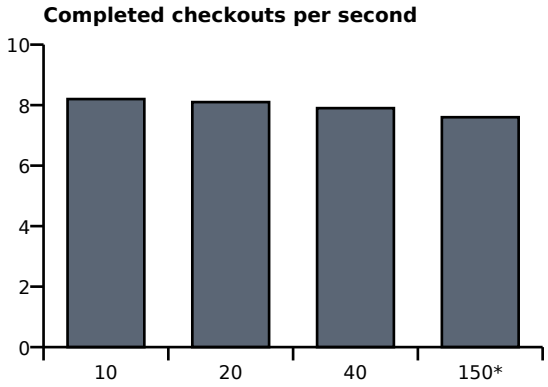
3. Capacity

Estimate: about **8 completed checkouts per second**, serving **~10 concurrent simulated buyers** with p95 under 2 s, inside the 0.5 vCPU / 512 MB API limit. Above that, extra requests are turned away with 503 + Retry-After and admitted buyers stay fast. On the shared production VM we expect **6–8 checkouts/s**.

3.1 Our load test

- **Tool:** `ops/loadtest/checkout.mjs` (Node, no dependencies). Each simulated buyer loops: `GET /v1/payment-links/:token` then `POST /v1/payment-links/:token/pay` with an Idempotency-Key and a simulated card.
- **Method:** API in a container with the judging limits (`--cpus 0.5 --memory 512m`, `NODE_ENV=production`, pool of 5, `LOAD_SHED_MAX_INFLIGHT=10`). Postgres outside the limit. The simulator adds 100–400 ms acquirer latency and 2% acquirer errors. A shed request is retried with the same link and Idempotency-Key. 30 s per run at 10, 20, 40 and 150 buyers.
- **Correctness check:** after each run, the test checks no payment link was paid twice.





* 150 buyers with load shedding at 10 in-flight: 4,111 requests shed with 503, then retried.

Buyers	Checkouts /s	p50	p95	p99	Shed (503)	Errors	Double payments
10	8.2	1.21 s	1.65 s	1.82 s	0	0	0
20	8.1	2.42 s	3.15 s	3.41 s	0 (limit was 40)	1 acquirer	0
40	7.9	4.65 s	5.71 s	6.90 s	0 (limit was 40)	7 acquirer	0
150	7.6	1.20 s	1.89 s	2.20 s	4,111	3 acquirer	0

3.2 Where the bottleneck is

The API's **0.5 vCPU**. Throughput stays flat at about 8/s from 10 to 150 buyers, and memory stays around 100 MB of 512 MB. Without shedding, extra buyers only add queueing (the 20 and 40 rows). The simulated acquirer's 100–400 ms is per request, not a shared limit. Postgres was outside the cap in the test; in production it shares the VM's single vCPU, which is why we quote 6–8/s.

3.3 Turning away excess load and recovering

- **Load shedding** (`apps/api/src/plugins/loadShed.ts`, tested): above `LOAD_SHED_MAX_INFLIGHT` (10) concurrent requests, the API answers `503` + `Retry-After: 1` immediately. Health checks are exempt. We first set the limit to 40; the measurements showed p95 climbing to 5.7 s, so we lowered it to 10.
- **Safe retries:** a shed request did no work, and the Idempotency-Key makes a retried pay call return the original result, never a second charge.
- **No double charge under contention:** a payment link is reserved with a token before the acquirer call. An attempt that loses the reservation after charging is refunded automatically.
- **Recovery:** systemd restarts the API within ~2 s after a crash or OOM kill. Postgres transactions mean no partial writes survive a crash.

4. Findings report

One entry per finding, ranked by impact within each group. **Status**: Fixed (merged, with proof), Mitigated (exposure stopped, root cause still open), Open. Commits: round 1 `4a68dc9`, round 2 `5c3fa2d`, deployment `150a8f8/056a82a`, this round `e010e18`. Detailed deployment log: `bugs.md`; detailed audit notes: `docs/AUDIT-FP-300.md`.

4.1 Findings

Money and settlement

F-01 Payouts ignore refunds: merchant overpaid, reconciliation does not tie out [Open]

root cause RC-5

Symptom	Demo day 2026-10-30: two captures (12.500 + 20.000 BD), one full and one partial refund (17.500 BD back to cardholders). The batch paid out **31.769 BD**, as if no refund happened. Dashboard Reconciliation shows a non-zero gap.
Found / reproduced	Running the judges' demo script locally (capture → full + partial refund → `settlement run-batch --date=2026-10-30`), then reading `settlement_batches` and `ledger_entries`. Screenshots in section 5.
Root cause	`CloseBatch` selects only `captured/partially_captured` events. Refunds post correct ledger legs (merchant_payable debited 17.500, fee reversal credited 0.393), but nothing deducts them from a payout. The 2026-06-02 eng sync deferred this to settlement v2 ("v1 can subtract refunds in the report and leave the batch alone"). v2 is not live, so the deferral became a bug.
Fix	**Not applied**: see decision D-1. Proposed: include refund events in the batch window as negative lines with their fee reversals, and carry a negative net forward (or hold the payout) instead of paying it.
Verified	Ledger after the batch: merchant_payable = **−17.107 BD** (credits 32.893, debits 50.000) = refunds 17.500 − fee reversals 0.393. The batch paid 32.500 − 0.731 = 31.769.
Existing data	Every batch paid after a refund is overpaid by refunds − fee reversals. Find them: `SELECT merchant_id, SUM(CASE direction WHEN 'credit' THEN amount_fils ELSE -amount_fils END) FROM ledger_entries WHERE account_id='merchant_payable' GROUP BY 1 HAVING SUM(...) < 0`. Needs a finance-approved clawback, not an automatic rewrite.

F-02 Finance CSV adjustments posted 10x too small [Fixed] root cause RC-1, RC-3

Symptom	Importing `4.75` BD produced 475 fils instead of 4,750. Finance had reported it; the reply was "it is in the tests".
Found / reproduced	Read the parser and its tests: the tests expected the 10x-wrong values. Reproduced with the import test fixtures.
Root cause	A local parser assumed two decimal places (cents), but BHD has three (ADR-0001). A commit marked it "intentional".
Fix	`parseAdjustmentAmountFils` delegates to `parseBhdToFils`; corrected the test expectations in the same change. Promotional discounts no longer use `parseFloat * 100`.
Verified	`adminAdjustmentImport.test.ts` / `adjustmentImport.test.ts` fail on the old parser, pass now.
Existing data	Adjustments imported before the fix stay 10x too small in the ledger. Finance must re-post the difference per import file; we did not run a backfill.

F-03 Concurrent refunds could both pay out; refunds were 501 [Fixed] root cause RC-2

Symptom	`POST /v1/refunds` returned 501. The parked helper refunded against the authorized amount and had no lock, so two parallel refunds could each pass the "refundable" check.
Found / reproduced	Code read of `processRefund.ts`; new test fires parallel refunds at one transaction.
Root cause	Check-then-act with no row lock; refundable computed from the authorized amount, not captured − refunded.
Fix	Lock the transaction (`FOR UPDATE`), reserve the amount in a `pending` refund row (0073) before the acquirer call; refundable = captured − refunded − pending; fee reversal from the net fee, last partial reverses the exact remainder; `refund.succeeded` webhook; dashboard drawer.
Verified	`routes/refunds.test.ts` (parallel refunds, over-refund, cross-merchant, fee sum). Live demo: fee reversals 281 + 112 fils at 2.25% (half-to-even).
Existing data	None. No refunds existed before (the route was 501).

F-04 Duplicate "payout on the way" webhooks (incident 2026-08-21) [Fixed] root cause RC-2

Symptom	Merchants received two `settlement.paid` notifications for one payout.
Found / reproduced	New Go test runs 8 enqueuers concurrently against Postgres; old code inserted more than one event.
Root cause	`EnqueuePayoutNotifications` ran `NOT EXISTS` then `INSERT` as two statements. The daemon tick and a CLI run raced. CHANGELOG 1.9.1 only told receivers to dedupe.
Fix	Migration 0076: partial unique index (one `settlement.paid` per batch) + `ON CONFLICT DO NOTHING`.
Verified	`payout_notify_test.go`: 8 concurrent enqueuers → exactly 1 event.
Existing data	Existing duplicate events are relabelled by 0076 (kept, because deliveries reference them).

F-05 Dashboard shows 245 bps; payments are charged 225 [Fixed] root cause RC-1, RC-5

Symptom	Finance memo: support quoted 2.45%, statements showed 2.25%.
Found / reproduced	Compared the display query with billing's `getEffectiveMdrBps`; an inherited test asserted 245 for exactly this case.
Root cause	Display ordered fee schedules by `created_at` (last entered); billing uses the latest `effective_from <= now()`. A backdated correction won the display. A "Claude says created_at is right" comment closed the question.
Fix	Display uses billing's rule (`fees/currentFeeScheduleDisplay.ts`); test corrected.
Verified	Test fails on old query (part of the 58 pass / 5 fail run with old code restored).
Existing data	Display only; no stored data changed.

F-06 Platform volume counter overflows; payout estimate drifts [Fixed] root cause RC-3

Symptom	Past ~2.1M BD platform volume the 15 bps tier discount would switch itself off for everyone; lifetime payout estimates drift for large merchants.
Found / reproduced	Migration 0049 says `INTEGER` and Go used `int32` (a comment claimed BIGINT); estimate column was `REAL` written from `float32`. Tests above `2^31` and `2^24`.
Root cause	Integer and float types too small for cumulative money.
Fix	Migration 0078: both BIGINT; Go int64; API reads BIGINT strings with `Number()`.
Verified	`platform_volume_test.go`, `payout_estimate_test.go` fail on old types.
Existing data	Values widened in place. Estimates already drifted stay approximate (monitoring only, not paid).

F-07 Go payouts used float32; split payouts did not sum [Fixed] root cause RC-3

Symptom	Net payout and split shares off by fils; child shares did not add up to the batch payout.
Found / reproduced	Code read; Go tests for split allocation.
Root cause	`NetPayableFils` computed in float32; splits rounded each share independently.
Fix	Integer arithmetic; largest-remainder allocation so shares sum exactly.
Verified	`go test ./internal/splits ./internal/settlement` pass against Postgres.
Existing data	Past batches keep their stored amounts (possible 1-fils differences); not rewritten.

F-08 MDR recalculation rewrote paid batches [Fixed] root cause RC-2

Symptom	Running Recalculate MDR over a month changed fees on batches already paid out.
Found / reproduced	Code read of `recalculate_mdr.go`.
Root cause	No status filter on the batches it updated.
Fix	Only `closed` (unpaid) batches are recalculated, under `FOR UPDATE`.
Verified	Go tests.
Existing data	Batches already rewritten before the fix cannot be distinguished automatically; finance to review recalculations run since July.

F-09 Admin "rebuild balances" disagreed with the CLI (FP-203) [Fixed] root cause RC-5

Symptom	Console button and `settlement rebuild-balances` gave different balances.
Found / reproduced	Two tests that seeded scheme export lines failed once the source was corrected.
Root cause	Button read scheme export lines; CLI read the ledger.
Fix	Both read `SUM(credit − debit)` of `merchant_payable` in the ledger; tests reseeded via ledger.
Verified	`refresh_test.go` passes.
Existing data	Balances recomputed from the ledger the next time either is run; no stored history changed.

F-10 Daemon's "today" used the UTC date [Fixed] root cause RC-3

Symptom	Between 21:00 and 24:00 UTC the daemon picked the wrong business day.
Found / reproduced	Code read against `BusinessDayWindow`.
Root cause	Raw UTC instant passed where a Bahrain calendar date was expected.
Fix	`settlement.BusinessDate(now)` first.
Verified	Scheduler test.
Existing data	None observed.

F-11 March sweep script posts twice when re-run [Fixed] root cause RC-2

Symptom	Finance: staging gap doubled after a second run.
Found / reproduced	Read script; comment claimed `entry_group_id` dedupes it, but it is a fresh `randomUUID()` each run.
Root cause	No idempotency guard.
Fix	Advisory lock + refuse if the sweep's ledger entries already exist.
Verified	`reconcile_march.integration.test.ts`.
Existing data	Staging has the doubled entries; reverse them there. Production was not run.

F-12 Refund report double-counted captures [Fixed] root cause RC-5

Symptom	`refund_merchant_summary` showed a payment refunded in two parts with its capture counted twice.
Found / reproduced	Query on a 2-part refund.
Root cause	View summed `captured_fils` once per refund row.
Fix	Migration 0077 aggregates per transaction first.
Verified	Reporting test.
Existing data	View only; no stored data.

F-13 Large transaction export could run out of memory or skip rows [Fixed] root cause RC-3

Symptom	Big merchants' exports risked OOM on 512 MB; the unused streaming path skipped rows.
Found / reproduced	Test with many rows sharing one timestamp.
Root cause	Route built the whole CSV in memory; streaming paged on `occurred_at` only.
Fix	Route streams; keyset paging on `(occurred_at, id)`.
Verified	`exportCsv.test.ts`.
Existing data	None (read-only).

Security and tenant isolation

F-14 The login code (OTP) was optional [Fixed] root cause RC-4

Symptom	After the password step, the pending-session cookie already worked on every merchant endpoint.
Found / reproduced	Called a merchant endpoint with the pending cookie; it returned data.
Root cause	`lookupSession` read the `live_sessions` view, which filters revoked/expired rows but not `authenticated`. Admin checked it; merchant never did.
Fix	`AND s.authenticated = true` (`auth/sessions.ts`).
Verified	`app.test.ts`: pending cookie → 401.
Existing data	Outstanding pending sessions stop working. No stored data changed.

F-15 Logging in to merchant B could open merchant A [Fixed] root cause RC-4

Symptom	With one email under two merchants, B's password produced a session in A.
Found / reproduced	Test with `owner@alpharetail.example` (member of two merchants).
Root cause	After checking the password for (merchant, email), login called `findOtpRecipient(email)`, which picks any merchant with that email (`LIMIT 1`).
Fix	Session created for the row whose password was checked.
Verified	`app.test.ts`.
Existing data	Sessions issued wrongly before the fix have expired; check audit logs for cross-merchant activity.

F-16 Rotating an API key left the old key working [Fixed] root cause RC-1, RC-4

Symptom	Admin "rotate" issued a new key; the old one still authenticated.
Found / reproduced	Test; the inherited test asserted a "grace period" no document defines.
Root cause	`rotateApiKeyAsAdmin` never set `revoked_at`.
Fix	Revoke the old key in the same transaction, reason `rotated`.
Verified	`admin.test.ts`.
Existing data	Keys rotated before the fix are still active. Revoke: any merchant with more than one active key from a rotation.

F-17 Refund and payment-link cancel looked up by id only [Fixed] root cause RC-4

Symptom	Merchant A could refund or cancel merchant B's transaction/link by id.
Found / reproduced	Cross-merchant test.
Root cause	Query not scoped to the caller's merchant.
Fix	Scoped by merchant.
Verified	`refunds.test.ts`.
Existing data	None found.

F-18 Admin impersonation could move money [Fixed] root cause RC-4

Symptom	Support impersonating a merchant could capture, void and refund.
Found / reproduced	Route review.
Root cause	Write routes did not reject impersonated actors.
Fix	`requireMerchantWriterActor` on every write; impersonation is read-only.
Verified	Route tests.
Existing data	Check admin audit export for impersonated writes.

F-19 OTP codes, link tokens and invite codes used Math.random() [Fixed] root cause RC-1

Symptom	Predictable secrets.
Found / reproduced	grep.
Root cause	Non-cryptographic RNG.
Fix	`node:crypto` via `lib/random.ts`.
Verified	Tests.
Existing data	Existing unexpired links/invites keep old tokens until they expire.

F-20 Webhooks signed over different bytes than were sent [Fixed] root cause RC-5

Symptom	Merchants' signature verification failed.
Found / reproduced	Recomputed the HMAC over the delivered body.
Root cause	Dispatcher signed one serialization and sent another.
Fix	`serializeWebhookBody` is used for both.
Verified	`dispatcher.test.ts`.
Existing data	None; old deliveries can be replayed from the archive.

F-21 Acquirer callbacks processed twice; payment links double-charge or accept late payment [Fixed] root cause RC-2

Symptom	Duplicate callbacks applied twice; a slow attempt could charge a link already paid; expiry trusted the buyer's `submittedAt`.
Found / reproduced	Code read; concurrency tests.
Root cause	Dedupe table `processed_events` never existed; no reservation on links; client clock trusted.
Fix	Dedupe on `callback_id` (0074); reservation token (0075) with automatic refund of the losing attempt; server-side expiry.
Verified	Tests + load test: 0 double payments.
Existing data	New constraints applied cleanly on existing data.

Deployment, data and UI

F-22 API site served the repository as static files [Mitigated] root cause RC-6

Symptom	`filspay-admin-login.txt` and `merchant-details.json` downloadable from the API host.
Found / reproduced	Requested the files by server IP: 200.
Root cause	Dilmun site created as a static site rooted at the checkout.
Fix	nginx is now a reverse proxy to 127.0.0.1:3000; files untracked and gitignored.
Verified	Both return 404 over HTTPS.
Existing data	**The exposed admin password and sandbox secret must be rotated** (still in git history).

F-23 Transaction page crashed after any refund [Fixed] root cause RC-1, RC-3

Symptom	Found today: after a successful refund the transaction page showed "Unexpected Application Error: toFils: expected a non-negative amount, got −5000". The refund had worked.
Found / reproduced	Playwright run of the refund demo against the local stack (screenshots, section 5).
Root cause	The timeline shows refunds as signed deltas; the dashboard's `bhd()` rejected negatives. A comment had flagged it; the reply was "amounts are never negative in the dashboard. ignore".
Fix	`bhd()` allows negatives (display only); commit `e010e18`.
Verified	New test fails on the old helper, passes now; page renders −5.000 BHD.
Existing data	None (display only). **fp-pay must be redeployed** for the live site.

F-24 Postgres 5432 open to the internet [Open] root cause RC-6

Symptom	Port 5432 reachable from outside; `pg_hba.conf` accepts `0.0.0.0/0`.
Found / reproduced	External port check.
Root cause	Default Dilmun firewall rule.
Fix	Not applied: deleting the firewall rule was blocked by our assistant's permission check and needs the Dilmun dashboard.
Verified	—
Existing data	Rotate the database password after closing it.

F-25 Production cannot deliver login codes [Mitigated] root cause RC-6

Symptom	No email/SMS sender; nobody could log in to the live site.
Found / reproduced	Live login attempt; notifier logs only "otp code generated".
Root cause	`auth/notifier.ts` is a stub.
Fix	`OTP_DEV_EXPOSE=true` on the demo shows the code on the login page (password alone logs in).
Verified	Demo admin and merchant log in over HTTPS.
Existing data	None. Remove the flag before real money.

F-26 Fresh database could not be built; 20+ ops defects [Fixed] root cause RC-6

Symptom	Migrations 0054, 0055, 0072 referenced missing tables/columns; dev key 34 bytes; compose ran production mode; deep links 404; bundle pointed at localhost; no DNS/HTTPS (bugs.md #4–#24).
Found / reproduced	Fresh migrate; local compose; live checks.
Root cause	Untested migrations and config; docs that contradicted the code.
Fix	Fixed each (see `bugs.md`); `setup.sh` fails loudly.
Verified	72 migrations apply on an empty Postgres (re-run today); 453/453 tests.
Existing data	None.

4.2 Root causes

	Root cause and the decision behind it	Findings
RC-1	**Trusting generated code, comments and tests over the ledger.** The "Order of the Prompt" rules told assistants to treat tests and "intentional" comments as truth. Tests were written to assert the bugs (10x CSV, 245 bps, key grace period), and pasted AI explanations closed questions. We replaced the rules (`CLAUDE.md`) and fixed each test in the same change as its defect.	F-02, F-05, F-16, F-19, F-23
RC-2	**Check-then-act without a lock or a database-level idempotency guard.** The design relied on "it runs once" (one daemon, one click). In practice daemon and CLI overlap, buyers double-submit and acquirers retry. Fix pattern: `FOR UPDATE`, reservation rows, unique indexes + `ON CONFLICT`, client Idempotency-Keys.	F-03, F-04, F-08, F-11, F-21
RC-3	**Money not typed as integer fils end to end.** `packages/money` was correct, but code around it used cents-style parsing, float32, int32 and REAL. Fix pattern: parse/format only through the audited helpers; BIGINT/int64 everywhere money accumulates.	F-02, F-06, F-07, F-10, F-13, F-23
RC-4	**Authentication and tenancy checked in some places, not all.** Lookups by id or email without the caller's merchant; session state not checked on one plane; impersonation not separated from writes.	F-14 to F-18
RC-5	**Two sources of truth for one number.** Display vs billing, button vs CLI, signed bytes vs sent bytes, batch vs ledger. The decision to "fix it in v2" left v1 with two answers.	F-01, F-05, F-09, F-12, F-20
RC-6	**Infrastructure configured by hand, with docs that claimed more than the code did.**	F-22, F-24, F-25, F-26

4.3 Decisions and open risks

#	What we chose not to change or run	Why	Safeguard we propose
D-1	Net refunds out of settlement payouts (F-01)	Changes who is paid how much; finance owns the rule (net vs clawback vs hold), and a payout change on deadline day is riskier than the bug. Found today.	Until fixed: hold payouts for merchants whose ledger payable is negative (query in F-01), and add a reconciliation alert when the dashboard gap ≠ 0. Then implement refunds as negative batch lines with carry-forward, with a test that ties the batch to the ledger.
D-2	Backfill historic data (CSV 10x, rotated keys, MDR rewrites)	Production data is demo data; real corrections need finance sign-off per file/merchant.	Queries listed per finding; run in a transaction with a dry-run report first.
D-3	Purge secrets from git history	Needs a force-push that rewrites everyone's history; owner's decision.	Rotate the admin password, sandbox acquirer secret and DB password now; history rewrite optional after.
D-4	Close Postgres 5432 (F-24)	Firewall change is in the Dilmun dashboard; our assistant's permission rules blocked it.	Delete firewall rule #4; remove 0.0.0.0/0 from pg_hba.conf.
D-5	Real OTP delivery (F-25)	Needs an email/SMS provider account.	Keep the demo on simulated money only; remove `OTP_DEV_EXPOSE` before any real use.
D-6	Never run `npm run seed` on the server	It truncates merchants, transactions and the ledger.	Documented; `.claude/settings.json` asks before seed.

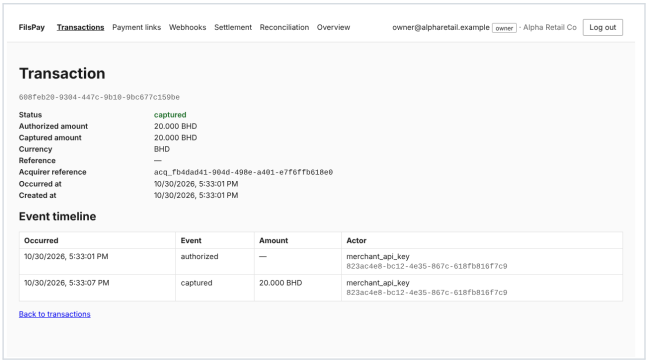
Still unresolved: scheme rounding residue posting (ADR-0002); a reaper for refunds stuck in `pending` after a crash; rate limits on OTP verify/resend; admin session token rotation after OTP (FP-198); Dilmun auto-deploy not firing (bugs #17); settlement "next check-in" log message (bugs #21).

5. UI and UX changes

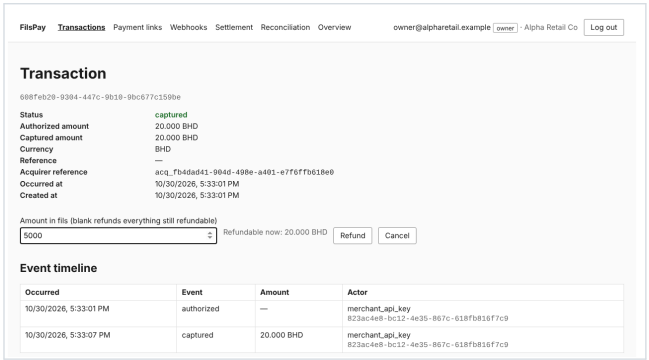
Screenshots taken today with Playwright against the stack running locally on a freshly migrated and seeded Postgres. "Before" is the dashboard at commit `698087e` (before our changes), "after" is the current code. Same API and data for both. No video recording.

5.1 Refund control on the transaction page

What: the RefundDrawer was parked and imported nowhere, so merchants had no way to refund. It is now wired to `POST /v1/merchants/:id/refunds`, shows the refundable amount (captured − refunded), validates whole fils, asks for confirmation and reuses one Idempotency-Key per attempt. **Why it mattered:** refunds are the core FP-300 feature; the parked version also posted to the wrong route and created a new idempotency key per click.



Before: captured payment, no refund control.



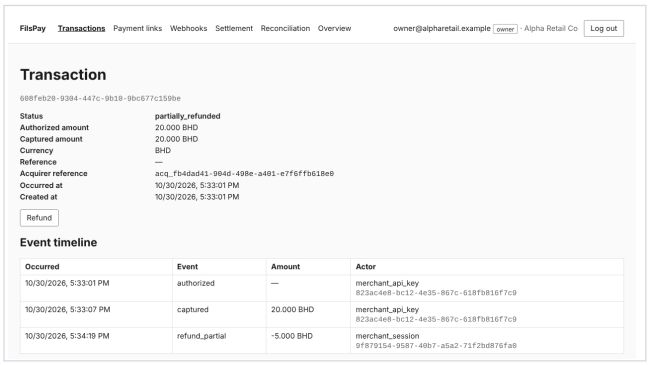
After: Refund drawer, partial refund of 5000 fils, "Refundable now: 20.000 BHD".

5.2 Transaction page no longer crashes after a refund

What: refund events are shown as signed amounts in the timeline. **Why it mattered:** without this, every refunded payment showed an error page (finding F-23). This is the page the refund demo ends on.



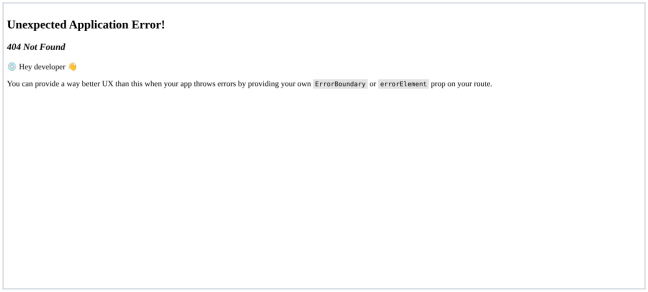
Before (today, pre-fix): error after a successful 5.000 BD refund.



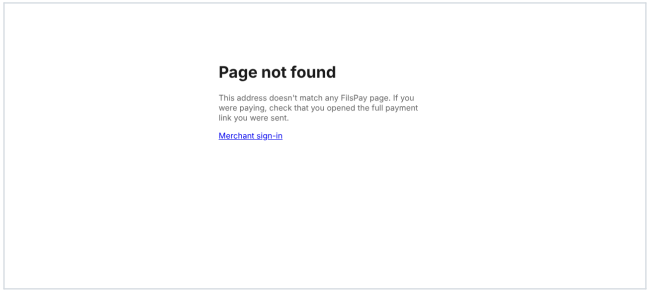
After: status partially_refunded, timeline shows −5.000 BHD.

5.3 Site root and unknown paths

What: / redirects to the dashboard (login when signed out); unknown paths show a "Page not found" page with a link to sign in. **Why it mattered:** the site's own address and any mistyped payment link showed React Router's developer error screen to buyers.

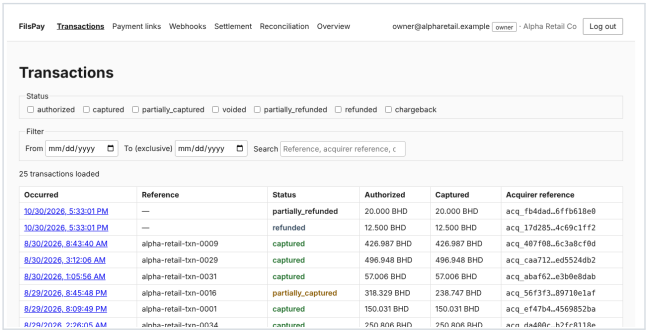


Before: /dashbord (typo) shows a developer error.

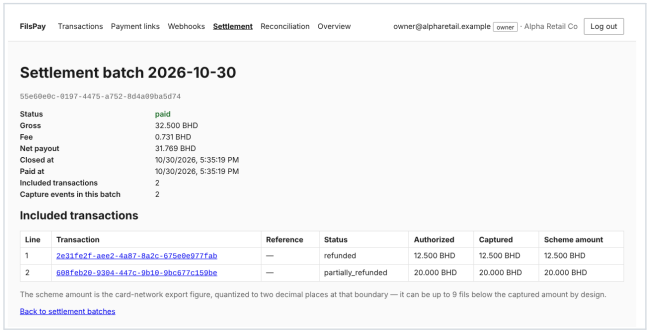


After: friendly Page not found.

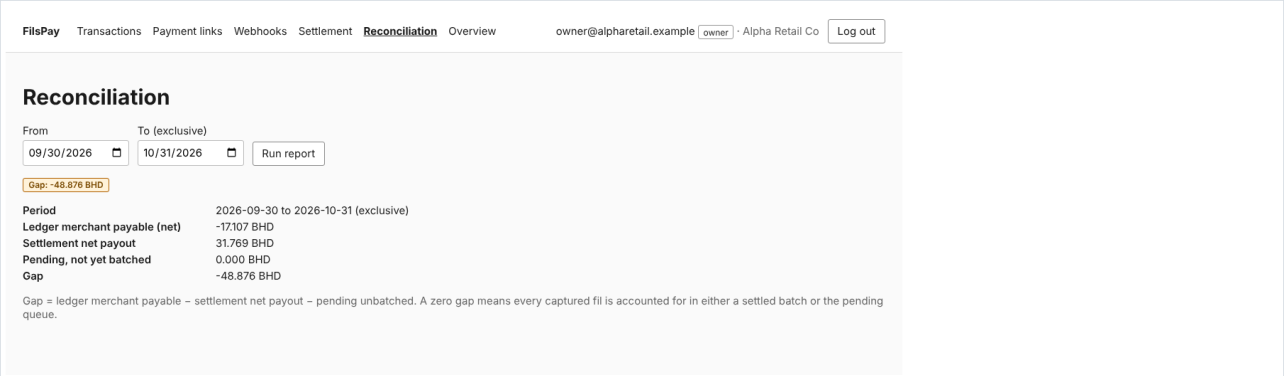
5.4 Refund and settlement views after the demo



Transactions list: refunded and partially_refunded with correct captured amounts.



Settlement batch 2026-10-30: gross 32.500, fee 0.731, net 31.769 (refunds not netted, F-01).



Reconciliation: the gap is non-zero after the refunds, which is the visible symptom of F-01.

Other UI changes without screenshots: the admin console's fee display now shows the rate billing applies (F-05); impersonation hides write actions server-side (F-18); the login page shows the code on the demo deployment only (F-25, a mitigation, not a feature).

6. AI usage

6.1 Setup

What	Where in the repo	Purpose
Agent instructions	`CLAUDE.md` (single source), `AGENTS.md`, `.cursorrules`	Replaced the "Order of the Prompt" rules: integer fils, Bahrain day, node:crypto, merchant scoping, locks, idempotency, and "verify, don't defer".
Permission rules	`.claude/settings.json`	Allow tests/typecheck; ask before seed, push, setup.sh, psql; deny reading .env and committed-secret files, deny force push.
Custom commands	`.claude/commands/verify-fix.md`, `check-live.md`, `money-review.md`	/verify-fix: failing test first, then fix. /check-live: prove server state. /money-review: review a diff against the money rules.
Helper scripts	`ops/dilmune/setup.sh`, `ops/loadtest/checkout.mjs`, `create-merchant.cjs`, `scripts/docs/build_report_pdf.py`	Redeploy, load test, merchant signup, report generation.
Tools	Claude Code (desktop app and claude.ai/code), Playwright, local Postgres 16, Go toolchain, Dilmun console	The assistant ran tests and queries itself; every claim needed evidence.

Rule we worked by: **the AI can propose; only evidence closes an item** (a test that fails on the old code, a database query, or a request to the live server). The permission rules and commands were committed on 30 October to codify how we had been working; the instruction files date from round 1.

6.2 Excerpt 1: an inherited test that asserted the bug

Finding	F-05 (245 vs 225 bps) and F-16 (key rotation)
Prompt	Work through the finance memos and handwritten notes; for each, find the root cause and prove it. (Summarised; we did not keep the exact wording.)
Context / tools	Repository, finance memo text, Postgres test database; the AI ran the test suite itself.
AI answer (relevant part)	"The second test is exactly Fatima's case (225 entered, 245 backfilled later), and it asserts the wrong answer. I'll fix the query to match billing and correct that test." It also flagged the inherited comment `// FH: asked Claude, it said created_at is right` as an unverified claim.
How we verified	We read billing's own query (`effective_from <= now() ORDER BY effective_from DESC`) and the memo, then **swapped the old code back in**: `✖ shows the rate in effect...`, `✖ API key rotation ... revokes the previous key`, 58 pass / 5 fail. With the fix: 377 pass / 0 fail.
What we changed	Display query and test (F-05); key rotation and its "grace period" test (F-16). The same method caught the BIGINT comment (F-06) and the sweep comment (F-11).

6.3 Excerpt 2: the AI was wrong, and we caught it

Finding	F-22 / deployment: rotating the exposed admin password
Prompt	"Ends in |t" (us, pointing out that the reset output did not show the expected row).
Context / tools	Dilmun console command history, read-only SQL against the database the API uses.
AI answer (relevant part)	First the AI theorised that the seed script was overwriting the password. Then it disproved its own theory: one admin row, created 06:49, and no cron, timer or pm2 job runs the seed. "The API reads that same database, but the reset never changed the password (f)."
How we verified	We required state, not output: the reset counted only when the command appeared in the console history with `UPDATE 1 … |t`. Same pattern for the API restart: it printed `SHADOWSTACK_REMOVED` but uptime was still 6,600 s; after the rerun uptime was 35 s.
What we changed	Re-ran the reset properly. We stopped accepting "printed success" and wrote /check-live to make that the default.

6.4 Excerpt 3: the AI's own report was wrong; a browser run showed it

Finding	F-23 (refund crash) and F-01 (payouts ignore refunds)
Prompt	The submission guidelines, pasted in full, including: "Live demo: a full and a partial refund end to end, with correct amounts in the dashboard and settlement report. Have screenshots ready."
Context / tools	Local Postgres 16 (migrated + seeded), API, settlement daemon and both dashboards; Playwright for screenshots; `psql`.
AI answer (relevant part)	Earlier the same day the AI had written a report saying refunds worked "end to end". Taking the demo screenshots, it reported: "the screenshots caught a real bug. After a refund, the transaction detail page crashes with 'toFils: expected a non-negative amount, got −5000'." Then, after running settlement: "the day's batch paid out 31.769 BD, as if the refunds never happened."
How we verified	Wrote a test for `bhd(-5000)`; it failed with the old helper (8 pass / 1 fail) and passed with the fix (9/9). Queried the ledger: merchant_payable −17.107 BD = 17.500 refunded − 0.393 fee reversal; the batch row showed net 31.769.
What we changed	Fixed the display helper (e010e18). Did **not** let the AI change payout logic on deadline day: recorded F-01 and decision D-1 instead.

Appendix

A. Test output (fresh Postgres 16, 30 Oct 2026)

```
$ npm run migrate            # empty database
migrate: 72 migration(s) applied, 0 already up to date
$ npm test
# tests 4    # pass 4    # fail 0      (packages)
# tests 40   # pass 40   # fail 0
# tests 385  # pass 385  # fail 0      (apps/api)
# tests 11   # pass 11   # fail 0      (apps/reporting)
# tests 8    # pass 8    # fail 0      (apps/web)
# tests 5    # pass 5    # fail 0      (scripts)
$ cd apps/settlement && DATABASE_URL=... go test ./...
ok  internal/config  internal/internalapi  internal/ledger  internal/money
ok  internal/scheduler  internal/settlement  internal/splits
```

B. Refund demo evidence (local, 2026-10-30)

```
POST /v1/transactions 12500 -> captured     POST /v1/transactions 20000 -> captured
dashboard: full refund 12500, partial refund 5000

transaction | amount | captured | status             | refund | fee_reversed
2e31fe2f... |  12500 |    12500 | refunded           |  12500 |          281
608feb20... |  20000 |    20000 | partially_refunded |   5000 |          112

$ settlement run-batch --date=2026-10-30 --merchant=9972daab...
batch=55e60e0c... gross=32500 fee=731 netPayout=31769

ledger merchant_payable for these rows: credits 32893, debits 50000  -> -17107
```

C. Migrations added

Migration	Purpose
0073	Pending refund reservation
0074	Acquirer callback dedupe on callback_id
0075	Payment-link reservation token
0076	One settlement.paid event per batch
0077	Refund summary counts each capture once
0078	BIGINT running totals
0054 / 0055 / 0072	Fixed so a fresh database builds

D. Reproduce

```
npm ci && npm run migrate && npm run seed          # local Postgres, never production
npm test && (cd apps/settlement && go test ./...)
node ops/loadtest/checkout.mjs --api http://127.0.0.1:3300 --users 150 --seconds 30
```